# DocuRAG — Implementation Plan

**AI-Powered Document & Image Search & Analysis (Multimodal RAG)** SOA University × Cognizant Hackathon · Use Case #3 · Mentor: Bindiya Paswan Build window: 5–12 Oct 2026 · **Evaluation: 13 Oct 2026** Compute: Google Colab (T4 GPU) · Backend: Django · Order of work: core RAG first, frontend/backend after

## 1. Problem & Objective

**Problem (from the use-case table):** Teams search large PDF and image repositories manually or with keyword-only tools. Exact-match search misses differently phrased information, while scanned pages, charts, and embedded images remain difficult to search.

**Objective (from the use-case table):** Create a multimodal RAG solution that ingests PDFs and images, retrieves relevant text and visual context, and generates grounded answers with **sources, confidence, and response-time information.**

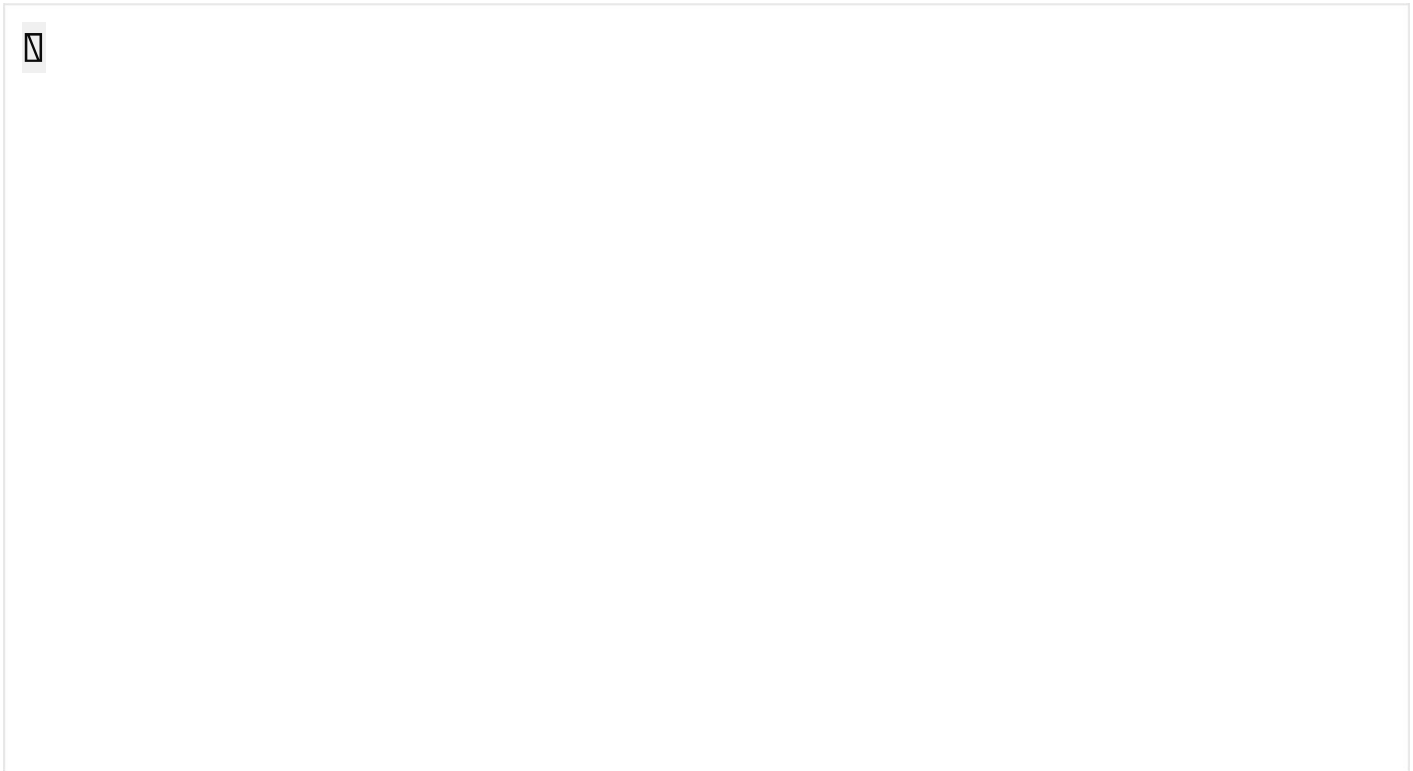
### Requirement → Feature traceability

| Requirement | How DocuRAG satisfies it | Tier |
|---|---|---|
| Ingest PDFs and images | Upload endpoint + ingestion pipeline | MUST |
| Different wording (semantic) | Dense embeddings + hybrid retrieval | MUST |
| Scanned PDFs | Text-density check → OCR fallback | MUST |
| Tables | Table extraction → markdown chunks | MUST |
| Charts / diagrams / embedded images | Vision-LLM captions + extracted values, embedded as searchable chunks | MUST |
| Retrieve text **and visual** context | Unified index with `content_type` = text / table / image | MUST |
| Grounded answer from LLM | Evidence-only prompt, JSON-structured output | MUST |

| Requirement | How DocuRAG satisfies it | Tier |
|---|---|---|
| Sources / citations | Doc + page + chunk + thumbnail per citation | MUST |
| Confidence | Composite score (retrieval + verifier + citation coverage) | MUST |
| Response time | Per-stage timing returned with every answer | MUST |
| Multi-document reasoning | Retrieve across docs, answer cites multiple sources | SHOULD |
| Agents + MCP | LangGraph orchestrator + MCP tool servers + verifier | SHOULD (after MUST is demo-ready) |
| Evaluation numbers | Small gold Q&A set, hit-rate / faithfulness | SHOULD |

**Rule for the 5 days:** a reliable demo of every MUST beats an unfinished agent system. Agents/MCP are layered on top of a pipeline that already works without them.

## 2. Architecture



**Key design decision:** the RAG engine lives in a plain Python package (`docurag/`) with **no Django dependency**. It is built and tested in Colab notebooks first, then Django simply imports it. This matches the "core RAG first, backend/frontend later" order and means the demo can fall back to a notebook if the web layer breaks.

## 3. Tech Stack

| Layer | Choice | Why |
|---|---|---|
| PDF parsing | **PyMuPDF (fitz)** | Fast text, page render, embedded image extraction |
| Tables | PyMuPDF `find_tables()` / pdfplumber | Convert to markdown so they stay searchable |
| OCR | Tesseract or EasyOCR (GPU) | Scanned pages and image uploads; Gemini vision as fallback for hard pages |
| Image/chart understanding | **Gemini multimodal (Flash tier)** | Captions + chart value extraction without heavy local VRAM use |
| Text embeddings | `BAAI/bge-small-en-v1.5` (fast) or `bge-m3` (better, heavier) | Runs comfortably on T4 |
| Image embeddings (optional) | CLIP / SigLIP | Direct text→image retrieval as a second signal |
| Vector store | **FAISS** (flat inner-product, cosine via normalised vectors) | Zero setup in Colab; corpus is small |
| Keyword search | `rank_bm25` | Handles IDs, codes, exact terms (e.g., "M-204") |
| Fusion | Reciprocal Rank Fusion (RRF) | Simple, robust hybrid merge |
| Reranker | `BAAI/bge-reranker-base` | Big precision gain, fits T4 |
| Answer LLM | Gemini (current Flash model via API) | Multimodal, fast, free-tier friendly |
| Local fallback | Qwen2-VL / Qwen2.5-VL small, 4-bit | Offline backup if API is down or rate-limited |
| Orchestration (stretch) | LangGraph | Explicit graph: retrieve → generate → verify → retry |

| Layer | Choice | Why |
|---|---|---|
| Tool layer (stretch) | MCP Python SDK (FastMCP) | Expose retrieval/document tools to the agent |
| Backend | **Django + Django REST Framework** | Your chosen backend |
| Frontend | Django templates + Tailwind (CDN) + HTMX/vanilla JS | Fastest path to a polished UI in the time left; React only if time remains |
| Exposure from Colab | `cloudflared` or ngrok tunnel | Demo URL if the app runs in Colab |

API keys (Gemini, ngrok) go in Colab Secrets / `.env`, never in the repo.

## 4. Repository Structure

```
```

```
docurag/
├── docurag/                  # core RAG engine (no Django imports)
│   ├── config.py             # models, thresholds, paths
│   ├── ingest/
│   │   ├── pdf_loader.py     # per-page text, tables, images
│   │   ├── ocr.py            # scanned-page + image OCR
│   │   ├── vision.py         # Gemini caption + chart extraction
│   │   └── chunker.py        # structure-aware chunking
│   ├── index/
│   │   ├── embedder.py
│   │   ├── vector_store.py   # FAISS wrapper + id mapping
│   │   └── bm25_store.py
│   ├── retrieve/
│   │   ├── hybrid.py         # dense + BM25 + RRF
│   │   ├── rerank.py
│   │   └── filters.py        # doc / type / page filters
│   ├── generate/
│   │   ├── prompts.py
│   │   ├── answer.py         # grounded generation (JSON out)
│   │   ├── verify.py         # evidence verifier
│   │   └── confidence.py
│   ├── agents/               # stretch
│   │   ├── graph.py          # LangGraph workflow
│   │   └── mcp_servers/
│   │       ├── document_server.py
│   │       └── retrieval_server.py
│   ├── pipeline.py           # ingest_file(), ask() — single entry points
│   └── timing.py             # stage timers
├── backend/                  # Django project
│   ├── manage.py
│   ├── config/               # settings, urls
│   └── api/                  # models, serializers, views
├── frontend/                 # templates + static
├── notebooks/
│   ├── 01_ingest_and_index.ipynb
│   ├── 02_retrieval_and_answer.ipynb
│   └── 03_eval.ipynb
├── data/
│   ├── raw/                  # chosen public documents
│   └── eval/gold_qa.json
```

```
├── storage/                # uploads, page renders, FAISS index
├── requirements.txt
└── README.md
```

# 5. Dataset (no dataset is provided — assemble your own)

Aim for **8–12 public documents (~150–300 pages total)** that deliberately cover every modality:

| Need | Example source type |
|---|---|
| Text-heavy report | Public annual report / sustainability report / government report |
| Table-heavy | Financial statements, statistical bulletins |
| Chart-heavy | Industry or market reports (WHO / World Bank / RBI-style publications) |
| Diagram/technical | Product manual, engineering/technical documentation |
| **Scanned PDF** | Print an existing PDF page → scan/photo → save as PDF, or rasterise a PDF to images (no text layer) |
| Standalone images | Chart screenshots, diagrams, a photographed table |

Choose a **single coherent theme** (e.g., a company/industry domain) so cross-document questions make sense. Only use openly licensed or publicly published documents. Keep a `data/SOURCES.md` listing each file and its origin.

# 6. Core Pipeline (build in Colab first)

## 6.1 Ingestion (`ingest/`)

```
```

```
file ──► type check ─┬─► PDF ──► per page:
                     │        ├─ extract text (PyMuPDF)
                     │        ├─ if chars < ~50 → render page @200dpi → OCR
                     │        ├─ extract tables → markdown
                     │        └─ extract embedded images/charts → save PNG
                     └─► Image ──► OCR + vision caption
```

Per-page decisions:

1. **Text page:** keep text; also render a page thumbnail for the viewer.
2. **Scanned page:** OCR; if OCR confidence is low or the page is chart-like, send to Gemini vision for transcription.
3. **Images/charts:** skip trivial ones (logos, < ~150px, tiny aspect ratios). For the rest, call vision with a structured prompt:

```
Describe this figure for search. Return JSON:
{ "type": "bar_chart|line_chart|table|diagram|photo|other",
 "title": "...", "caption": "...",
 "key_values": [{"label": "...", "value": "...", "unit": "..."}],
 "relationships": "for diagrams: A -> B -> C",
 "visible_text": "..." }
```

The JSON is flattened into a text chunk ( `content_type="image"` ), keeping `key_values` so numeric questions about charts are answerable.

### 6.2 Chunking ( `chunker.py` )

- Text: ~300–500 tokens, split on headings/paragraphs, ~50-token overlap, never crossing a page without recording both pages.
- Tables: one chunk per table (split by row groups only if very large), markdown format, with nearest heading as context.
- Images: one chunk per figure, caption + extracted values + surrounding caption text from the page.
- Prepend a short context header to each chunk ( `Doc: X | Page: N | Section: Y` ) before embedding — this improves retrieval noticeably.

**Chunk schema (also the Django model):**

```
json
```

```
{
  "chunk_id": "c_000231",
  "document_id": "d_004",
  "source": "Production_Report.pdf",
  "page": 12,
  "content_type": "text | table | image | ocr",
  "section": "Q3 Results",
  "content": "...",
  "image_path": "storage/figs/d_004_p12_f1.png",
  "page_image_path": "storage/pages/d_004_p12.png",
  "ocr_used": false
}
```

### 6.3 Embedding & Indexing

- Normalise vectors; FAISS `IndexFlatIP` is enough at this scale (a few thousand chunks).
- Keep an `id → chunk_id` mapping, persisted with the index (`faiss.write_index` + JSON).
- Build BM25 over the same chunk text.
- Optional: CLIP/SigLIP embeddings of figure images in a second FAISS index; fuse as a third ranking list.
- Persist index to Google Drive so Colab restarts don't force re-ingestion.

### 6.4 Retrieval (`retrieve/`)

```
query ─► (optional) query rewrite / expansion
   ─► dense top-30  ┐
   ─► BM25  top-30  ├─► RRF fusion ─► top-20 ─► cross-encoder rerank ─► top-5..8
   ─► (img index)   ┘
```

- **RRF:** `score(d) = Σ 1 / (60 + rank_i(d))`.
- Guarantee **modality diversity**: if the top results are all text but the query mentions "chart / figure / table / diagram / trend", force-include the best image/table chunk.
- Filters: by document, by content type.
- Record the top reranker score — it feeds confidence.

### 6.5 Grounded Generation (`generate/`)

Send the question plus numbered evidence blocks (`[E1] Maintenance_Report.pdf p.18 (text): ...`). For image evidence, also attach the actual image to the Gemini call so the model can read the chart itself rather than trusting the caption alone.

Prompt rules:

```
Answer ONLY using the evidence blocks.
If the evidence is insufficient, say so and set "answerable": false.
Cite evidence ids after each claim, like [E2].
Return JSON: {"answer": "...", "citations": ["E1","E3"], "answerable": true}
```

Parse the JSON, map `E#` back to `(document, page, chunk, thumbnail)` for the response.

### 6.6 Verification (`verify.py`)

Second, cheaper LLM call: "*For each claim in the answer, is it supported by the cited evidence? Return per-claim supported/unsupported.*" Output: `support_ratio` (0–1). If `support_ratio < 0.7`, retry once with broader retrieval (top-12, include neighbouring pages); if still weak, return a low-confidence answer with a clear caveat.

### 6.7 Confidence (`confidence.py`)

Be transparent that this is a heuristic composite, not a calibrated probability:

```
confidence = 0.4 * norm(top_rerank_score)
           + 0.4 * support_ratio          # from verifier
           + 0.2 * citation_coverage      # share of answer sentences with citations
```

Show it as a percentage plus a label (High ≥ 80, Medium 50–79, Low < 50). If `answerable=false`, cap at 20% and show "insufficient evidence."

### 6.8 Response Time (`timing.py`)

A context-manager timer around each stage; return:

```
json
"timing": {"retrieval_ms": 420, "rerank_ms": 180, "generation_ms": 1650,
           "verification_ms": 700, "total_ms": 2950}
```

Display total in the UI with an expandable per-stage breakdown. This directly satisfies the "response-time information" requirement.

# 7. Agentic Layer + MCP (stretch — only after sections 6 and 8 work end to end)

**Principle:** agents decide *what to do*; MCP standardises *which tools exist*; the core pipeline functions do the real work. The agent layer wraps existing functions — it doesn't replace them.

## 7.1 MCP servers (FastMCP, stdio transport)

| Server | Tools |
| --- | --- |
| `document_server` | `list_documents()`, `get_page_text(doc, page)`, `get_page_image(doc, page)`, `get_figure(doc, fig_id)` |
| `retrieval_server` | `hybrid_search(query, k, filters)`, `search_images(query, k)`, `search_tables(query, k)`, `rerank(query, ids)` |

## 7.2 LangGraph workflow

```
```

Keep it to **one orchestrator graph with 4–5 nodes**. A graph trace (which nodes ran, which tools were called) is shown in an "Agent trace" panel in the UI — this is the visible proof of the agentic design.

**Fallback:** a config flag `USE_AGENTS=false` runs the linear pipeline from section 6. If the graph is flaky on demo day, switch it off.

# 8. Django Backend

## 8.1 Models

```
Document(id, filename, file, file_type, pages, status[queued|processing|ready|failed],
    uploaded_at, error)
Chunk(id, document FK, page, content_type, section, content, image_path,
   page_image_path, ocr_used)
QueryLog(id, question, answer, confidence, total_ms, timing_json,
    citations_json, created_at)
```

FAISS/BM25 hold vectors; Django holds metadata. The `faiss_id → Chunk.id` map ties them together.

### 8.2 API endpoints (DRF)

| Method | Endpoint | Purpose |
|---|---|---|
| POST | `/api/documents/` | Upload PDF/image, start ingestion |
| GET | `/api/documents/` | List docs with status, page count |
| GET | `/api/documents/{id}/` | Detail + chunk counts per type |
| GET | `/api/documents/{id}/pages/{n}/` | Page image for the viewer |
| POST | `/api/query/` | `{question, filters?}` → answer payload |
| GET | `/api/history/` | Past queries |
| GET | `/api/health/` | Index size, model status |

### 8.3 Query response contract

```
```

json

```
{
 "answer": "The defect rate was 21% in 2025 versus 12% in 2023, a 9-point increase. [E1]",
 "answerable": true,
 "sources": [
  {"id": "E1", "document": "Production_Report.pdf", "page": 12,
   "type": "image", "snippet": "...", "thumbnail_url": "/media/pages/d_004_p12.png",
   "score": 0.86}
 ],
 "confidence": {"score": 0.91, "label": "High", "support_ratio": 0.95},
 "timing": {"retrieval_ms": 420, "rerank_ms": 180, "generation_ms": 1650,
        "verification_ms": 700, "total_ms": 2950},
 "agent_trace": ["analyze_q", "retrieve:hybrid_search", "vision_read", "generate", "verify"]
}
```


### 8.4 Ingestion handling

Ingestion can take minutes. Use a **background thread** (or Django-Q / Celery only if already comfortable; a simple `threading.Thread` + status field is enough for a hackathon) and have the UI poll document status.

### 8.5 Running on Colab

Colab sessions are ephemeral and have no public IP:

1. Mount Drive; keep `storage/` and the FAISS index there.
2. Install requirements, load models once per session.
3. `python manage.py runserver 0.0.0.0:8000` in a background process.
4. Open a `cloudflared` /ngrok tunnel; add the tunnel host to `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS`.
5. Pre-ingest the demo corpus **before** the evaluation so the demo starts instantly.

**Safety net for the evaluation:** record a screen capture of a full successful run, and keep the notebook version runnable.

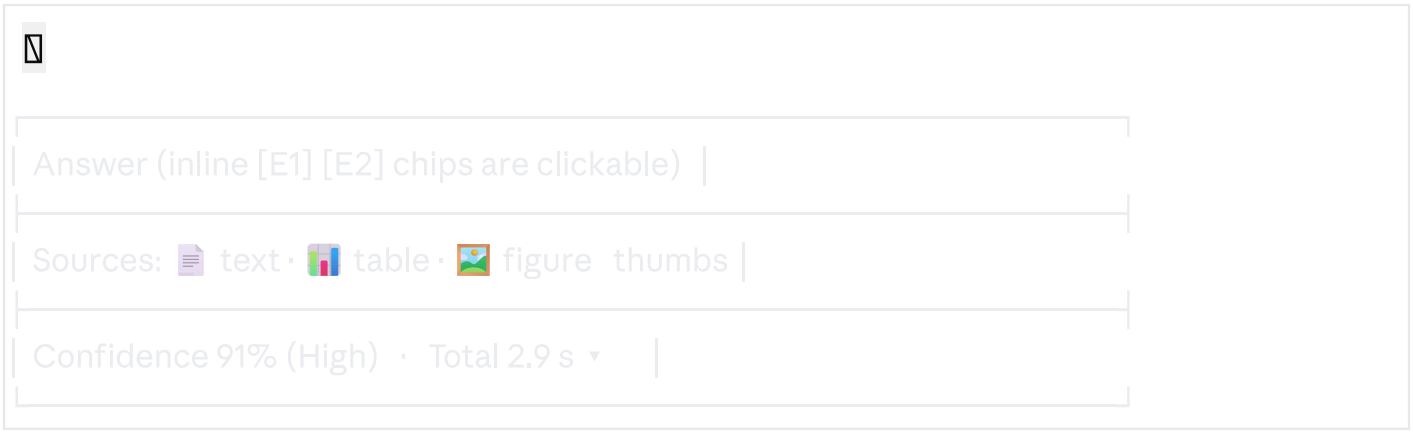
## 9. Frontend

### Pages

1. **Library** — drag-and-drop upload, per-file status (processing → ready), page count, counts of text / table / image chunks.
2. **Ask** — question box, answer card, sources panel, confidence badge, response-time pill.

3. **Viewer drawer** — click a citation → opens the exact page image with the cited area/figure highlighted (at minimum: show the page + the cited snippet; highlighting is a nice-to-have).
4. **History** — previous questions and answers.
5. **Agent trace** (if section 7 built) — collapsible step list.

### Answer card layout




Build the Ask + Library pages first; History and trace are last.

## 10. Evaluation (small but real numbers impress panels)

Create `data/eval/gold_qa.json` with **25–30 questions**, tagged by type:

| Type | Count | Example check |
|---|---|---|
| Paraphrase / semantic | 6 | Query wording not present in the document |
| Table lookup | 5 | Exact cell value |
| Chart reading | 5 | Value or trend from a figure |
| Scanned page | 4 | Fact only reachable via OCR |
| Diagram relationship | 3 | Component order/connection |
| Multi-document | 4 | Needs 2+ sources |
| Unanswerable | 3 | Must refuse with "insufficient evidence" |

Each entry: `question`, `expected_answer`, `gold_sources` (doc + page).

**Metrics to report:**

- **Retrieval hit@5** — gold page appears in top-5.
- **Answer correctness** — manual or LLM-judged, per type.
- **Faithfulness** — verifier support ratio average.
- **Refusal accuracy** — on unanswerable questions.
- **Latency** — median and p95 total time.
- **Ablation table** — keyword-only vs dense vs hybrid vs hybrid+rerank (this one slide proves the "exact-match search fails" claim with your own numbers).

## 11. Schedule (5 days left, 8–12 Oct; eval on 13 Oct)

Today is 8 Oct, so the plan is compressed. Each day ends with something runnable.

| Day | Goal | Deliverable at end of day |
|---|---|---|
| **Thu 8 Oct** | Colab env + dataset + ingestion | 8–12 docs collected; PDF text/table/OCR/image extraction working; chunks saved with metadata |
| **Fri 9 Oct** | Index + retrieval + generation | FAISS + BM25 + RRF + rerank; `ask()` returns grounded answer with citations, confidence, timings (notebook) |
| **Sat 10 Oct** | Django API + UI | Upload → ingest → ask → answer with sources working in browser, running through tunnel |
| **Sun 11 Oct** | Multimodal depth + verifier + agents | Chart/figure retrieval solid; verifier + retry; LangGraph + MCP wrapper (if the above is stable); eval set written and run |
| **Mon 12 Oct** | Freeze, polish, rehearse | Bug fixes only, pre-ingested corpus on Drive, backup video recorded, slides, 3 full demo run-throughs |
| **Tue 13 Oct** | Evaluation | Present |

**Cut order if time runs short** (cut from the bottom up): React rewrite → history page → agent trace UI → MCP servers → LangGraph (keep verifier as a plain function) → image-embedding index → page highlighting. Never cut: OCR, table + chart chunks, citations, confidence, response time.

## 12. Demo Script (≈5 minutes)

Rehearse the exact queries in advance, and keep one spare query per category.

## 13. Risks & Mitigations

| Risk | Mitigation |
|---|---|
| Colab session disconnects / GPU unavailable | Persist index + models cache on Drive; pre-ingest; use API-based vision/LLM so GPU is only needed for embeddings/rerank; backup video |
| Gemini API rate limits or outage | Cache vision captions per image hash; throttle ingestion; local small VLM fallback; keep a config switch |
| Ingestion too slow for live upload demo | Demo upload with a short 3–5 page file; everything else pre-ingested |
| OCR errors on poor scans | Use 200–300 dpi render; fall back to vision transcription on low-confidence pages |
| Chart values misread | Pass the actual image to the answer model, not just the caption; tell the user the source page for verification |
| Hallucination | Evidence-only prompt, `answerable` flag, verifier, retry-then-refuse |
| Over-scoping (agents/MCP) | Strict tiering; linear pipeline is the default; `USE_AGENTS` flag |

| Risk | Mitigation |
| --- | --- |
| Django on Colab networking (hosts/CSRF) | Set `ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS` from the tunnel URL on startup; test early on Day 3 |
| Per-person evaluation questions | Everyone on the team can explain every stage; keep the "why" for each choice in section 14 |

## 14. Likely Panel Questions — Short Answers

- **Why hybrid retrieval?** Dense handles paraphrase; BM25 handles IDs and exact terms; RRF merges them without score calibration. The ablation table shows the gain.
- **How do you handle charts and scanned pages?** Scanned pages are detected by text density and OCR'd; charts are converted by a vision model into searchable text with extracted values, and the original image is passed back at answer time.
- **How do you prevent hallucination?** Evidence-only prompting, an `answerable` flag, a claim-level verifier, and refusal when support is low.
- **What does confidence mean?** A composite of reranker score, verifier support, and citation coverage — a heuristic, not a calibrated probability.
- **Why MCP and agents?** MCP gives the agent standard tool interfaces over document and retrieval services; the agent decides which modality to search and whether to retry after verification. The pipeline still works without them.
- **How would it scale?** Swap FAISS for Qdrant/pgvector, move ingestion to a task queue (Celery), add per-user document access control, and cache embeddings and captions.

## 15. Definition of Done (checklist)

**Core**

- [ ] Upload PDF and image through the UI
- [ ] Scanned PDF correctly OCR'd and searchable
- [ ] Tables extracted and answerable
- [ ] Charts/figures captioned with values and retrievable
- [ ] Hybrid retrieval + rerank working
- [ ] Answers grounded, with doc + page citations
- [ ] Citation click opens the source page
- [ ] Confidence shown on every answer

- [ ] Response time (with stage breakdown) shown on every answer
- [ ] "Insufficient evidence" refusal works

**Advanced**

- [ ] Multi-document answers
- [ ] Verifier + retry loop
- [ ] LangGraph workflow + agent trace
- [ ] MCP document and retrieval servers
- [ ] Eval set run with a results table

**Delivery**

- [ ] Corpus pre-ingested and index stored on Drive
- [ ] README with setup and architecture diagram
- [ ] Backup demo video
- [ ] 3 full rehearsals completed
- [ ] API keys removed from the repo

## 16. First Hour Checklist (start now)

1. Create the repo and the folder skeleton from section 4.
2. Colab notebook `01_ingest_and_index.ipynb`: install `pymupdf pdfplumber sentence-transformers faiss-cpu rank_bm25 easyocr google-generativeai`.
3. Put the Gemini key in Colab Secrets; test one image → caption call.
4. Download 3 documents (one text, one chart-heavy, one scanned) to `data/raw/`.
5. Write `pdf_loader.py` and print per-page text/tables/images for one PDF.
6. Commit. Everything after this builds on that foundation.